

GitHub Enterprise Training Program

For a small delivery team (5-7 people) — from fundamentals to CI/CD and container deployment on Oracle Kubernetes Engine (OKE)

How to use this program

This is a **10-chapter, self-paced curriculum** designed for a team of 5-7 engineers who want to master GitHub Enterprise end-to-end: platform administration, day-to-day collaboration, automation with GitHub Actions, software supply-chain security, and a complete container build-and-deploy pipeline using **Podman** into **Oracle Cloud Infrastructure (OCI)** on **Oracle Kubernetes Engine (OKE)**.

Each chapter follows the same rhythm:

- **Objectives** — what you'll be able to do afterwards.
- **Concepts** — the ideas and how they fit together.
- **Hands-on lab** — a concrete exercise the whole team runs together.
- **Knowledge check** — questions to confirm understanding.

Suggested cadence: **one chapter per week** (10 weeks), each ~90 minutes of instruction plus a lab. A smaller team can compress this to 5 weeks by pairing chapters. Nominate one person as "**platform owner**" for the enterprise/org configuration chapters (1, 7, 10) and rotate lab facilitation so everyone gets hands-on with Actions and deployment.

Note on accuracy: GitHub and OCI both change their UIs and feature availability regularly, and some capabilities depend on your license tier (GitHub Enterprise Cloud vs Server, GitHub Advanced Security, Copilot). Treat the command snippets as a starting point and verify against the current official docs. Anything security-sensitive (secrets, OIDC, IAM policies) should be reviewed by your platform owner before running in a real tenancy.

Curriculum at a glance

#	Chapter	Theme	Primary tools
1	GitHub Enterprise Foundations	Platform & admin	Enterprise, Orgs, Teams, SSO
2	Repositories & the Collaboration Model	Source control	Repos, branch protection, rulesets
3	Pull Requests & Code Review	Quality gates	PRs, CODEOWNERS, reviews
4	Issues, Projects & Planning	Delivery flow	Issues, Projects v2, Milestones
5	GitHub Actions Fundamentals	Automation basics	Workflows, runners, secrets
6	Building CI Pipelines	Continuous integration	Matrix, caching, reusable workflows
7	Security & Supply Chain (GHAS)	Shift-left security	CodeQL, secret scanning, Dependabot
8	Container Builds with Podman	Image build	Podman, Buildah, OCIR, ghcr.io
9	Deploying to OCI OKE	Continuous deployment	OKE, OCIR, OIDC, kubectl/Helm
10	Operate, Govern & Scale	Advanced & capstone	Packages, Codespaces, Copilot, ARC

Chapter 1 — GitHub Enterprise Foundations

Objectives

- Explain the difference between GitHub Enterprise Cloud, Enterprise Server, and Enterprise Managed Users (EMU).
- Understand the enterprise → organization → team → repository hierarchy.
- Configure identity, membership, and baseline governance for a 5–7 person team.

Concepts

GitHub Enterprise comes in two hosting models. **Enterprise Cloud** is GitHub-hosted (github.com or a dedicated `ghe.com` data-residency instance). **Enterprise Server** is a self-hosted virtual appliance you run in your own infrastructure. For a small team, **Enterprise**

Cloud is almost always the right choice — no appliance to patch, and you get Actions, Codespaces, and Advanced Security as managed features.

The account hierarchy is a nesting of scopes:

- **Enterprise account** — the top-level billing and policy boundary. Enterprise-wide policies (SSO enforcement, allowed Actions, IP allow lists) cascade down.
- **Organizations** — where your repos, teams, and projects live. A team of 5–7 typically needs just **one organization**.
- **Teams** — named groups of members used to grant repository access in bulk and to request reviews. Teams can be nested (e.g. `@acme/backend` under `@acme/engineering`).
- **Repositories** — the unit of source, issues, actions, and packages.

Identity and access. With standard Enterprise Cloud you connect your identity provider (Okta, Entra ID, Google, etc.) via **SAML SSO** and provision users automatically with **SCIM**. With **Enterprise Managed Users (EMU)**, GitHub manages the user accounts entirely — members can only exist inside your enterprise and cannot use those accounts elsewhere. EMU gives tighter control but restricts collaboration with outside contributors, so choose deliberately.

Roles exist at each scope: enterprise owners, organization owners, org members, and per-repository roles (Read, Triage, Write, Maintain, Admin). Grant the **least privilege** that lets people do their job; for a small team, most engineers are org members with Write on the repos they work in, one or two people are org owners, and repository Admin is reserved for repo maintainers.

The audit log records who did what across the org/enterprise and is your first stop for security review and compliance. Learn where it lives early.

Hands-on lab

1. Create (or have your platform owner create) one organization inside your enterprise.
2. Configure SAML SSO against your IdP in test mode, then enforce it. Confirm each team member can sign in.
3. Create two teams — e.g. `maintainers` and `engineers` — and add members.
4. Set an organization-wide default repository permission and a base member role.
5. Open the **audit log** and locate the events generated by the steps above.

Knowledge check

- When would you choose EMU over standard SSO, and what do you give up?
- Which scope enforces SSO — repository, organization, or enterprise?
- Give an example of a task that needs Admin vs one that only needs Write.

Chapter 2 — Repositories & the Collaboration Model

Objectives

- Create well-structured repositories with sensible defaults.
- Choose a branching strategy suited to a small team.
- Protect important branches using branch protection rules and **rulesets**.

Concepts

A repository is more than code — it holds issues, pull requests, Actions, packages, releases, a wiki, and settings. Set it up well once: a clear `README.md`, a `LICENSE`, a `.gitignore`, a `CONTRIBUTING.md`, and issue/PR templates in `.github/`.

Branching strategy. For a team of 5–7, **trunk-based development** usually beats heavyweight GitFlow: everyone works on short-lived feature branches off `main`, merges frequently behind CI, and releases from `main` (optionally with release branches for hotfixes). This keeps integration pain low and pairs naturally with Actions.

Protecting `main`. GitHub offers two overlapping mechanisms:

- **Branch protection rules** — the classic per-branch settings: require pull requests, require passing status checks, require reviews, require signed commits, restrict who can push.
- **Rulesets** — the newer, more flexible successor. Rulesets can target branches *and* tags by pattern, can be layered, support "evaluate" (dry-run) mode, apply at the org level across many repos, and expose bypass lists. Prefer **rulesets** for new setups because they scale and audit better.

Typical protection for `main`: require a PR before merging, require at least one approving review, require status checks (your CI) to pass, dismiss stale approvals on new commits, and block force-pushes and deletions.

CODEOWNERS. A `.github/CODEOWNERS` file maps paths to owning teams/people so the right reviewers are requested automatically. Combined with "require review from Code Owners," it guarantees the database changes get a database reviewer, the pipeline changes get a platform reviewer, and so on.

Hands-on lab

1. Create a repository from a template, or add `README`, `LICENSE`, `.gitignore`, and a PR template.
2. Add a `CODEOWNERS` file assigning at least two path patterns to your two teams.
3. Create a **ruleset** on `main`: require PRs, one approving review, Code Owner review, and passing status checks; block force-push and deletion.

4. Prove it works: try to push directly to `main` and observe the rejection; open a PR and watch the required checks and reviewers appear.

Knowledge check

- Why prefer trunk-based development for a small team?
 - Name two things rulesets can do that classic branch protection cannot.
 - What does "dismiss stale approvals" protect you from?
-

Chapter 3 — Pull Requests & Code Review

Objectives

- Run an efficient, high-quality PR review workflow.
- Use draft PRs, review requests, suggestions, and merge strategies correctly.
- Tie reviews to your branch protection so quality is enforced, not hoped for.

Concepts

The **pull request** is where change is proposed, discussed, verified, and merged. A healthy PR lifecycle: branch → commit → open PR (draft if not ready) → CI runs → reviewers requested (manually or via CODEOWNERS) → review with comments and **suggested changes** → author addresses feedback → approvals gathered → required checks green → merge.

Reviews come in three verdicts: *comment*, *approve*, *request changes*. Use **suggested changes** (the `suggestion` code block) so reviewers can propose exact edits the author accepts with one click. Keep PRs **small** — a 200-line PR gets a real review; a 2,000-line PR gets a rubber stamp.

Merge strategies:

- **Merge commit** — preserves full branch history; can get noisy.
- **Squash and merge** — collapses the branch into one tidy commit on `main`; great for small teams that value a clean, linear history.
- **Rebase and merge** — replays commits without a merge commit.

Pick one default (squash is a common choice for small teams) and enforce it in repo settings.

Automation aids: auto-merge (merge automatically once checks pass and approvals land), required linear history, and **merge queue** (serializes merges so `main` is always tested in its final state — valuable once several PRs land per day).

Hands-on lab

1. Open a draft PR, then mark it ready for review.
2. As a reviewer, leave a **suggested change** and have the author accept it from the UI.

3. Enable **auto-merge** on a PR and watch it merge itself once checks pass and it is approved.
4. Compare the resulting `main` history under squash vs merge-commit strategies.

Knowledge check

- What is the difference between "request changes" and "comment"?
 - When is squash-and-merge preferable to a merge commit?
 - What problem does a merge queue solve that branch protection alone does not?
-

Chapter 4 — Issues, Projects & Planning

Objectives

- Track work with issues, labels, and milestones.
- Build a delivery board with **Projects (v2)** and automate it.
- Connect planning to code so status updates itself.

Concepts

Issues are the atomic unit of work — bugs, tasks, features. Enrich them with **labels** (type, priority, area), **assignees**, and **milestones** (time-boxed goals like a sprint or release). **Issue forms** (YAML templates in `.github/ISSUE_TEMPLATE/`) capture structured, consistent bug reports.

Projects (v2) is GitHub's flexible planning layer. A project is a collection of items (issues, PRs, and free-form notes) that you can view as a **board** (kanban), **table** (spreadsheet), or **roadmap** (timeline). You add **custom fields** — status, iteration, story points, priority — and slice with filters and grouping. Built-in **workflows** automate transitions: move an item to "In Progress" when a PR is opened, to "Done" when it's merged or the issue is closed.

Linking work to code. Reference issues from PRs and commits with closing keywords (`(Fixes #123)`) so merging the PR closes the issue automatically. This keeps the board honest with minimal manual bookkeeping.

Discussions provide a forum-style space for questions, RFCs, and announcements that don't belong in an issue tracker — useful even for a small team to preserve decisions.

Hands-on lab

1. Create an issue form template for bug reports.
2. Create a Project (v2) with `Status`, `Priority`, and `Iteration` fields; add a board and a roadmap view.
3. Add a built-in workflow that sets `Status = Done` when an item's PR is merged.

4. Open a PR with `Fixes #<issue>` and confirm the issue closes and the board updates on merge.

Knowledge check

- What are the three main Project (v2) views and when is each useful?
- How does a commit or PR automatically close an issue?
- Where would a design RFC live — an issue, a project, or a discussion?

Chapter 5 — GitHub Actions Fundamentals

Objectives

- Read and write workflow YAML confidently.
- Understand events, jobs, steps, runners, secrets, variables, and environments.
- Use the marketplace, caching, artifacts, and matrices.

Concepts

GitHub Actions is the built-in automation and CI/CD engine. A **workflow** is a YAML file in `.github/workflows/` triggered by **events** (`push`, `pull_request`, `schedule`, `workflow_dispatch`, `release`, and many more). A workflow contains **jobs**; each job runs on a **runner** and contains **steps**; a step either runs a shell command (`run:`) or invokes a reusable **action** (`uses:`).

Runners are either **GitHub-hosted** (ephemeral VMs — `ubuntu-latest`, `windows-latest`, `macos-latest`) or **self-hosted** (your own machines, useful for private networks or special hardware — see Chapter 10).

Secrets and variables. Store non-sensitive config as **variables** (`vars.*`) and sensitive values as **secrets** (`secrets.*`), defined at repo, environment, or organization scope. Secrets are masked in logs. Prefer **environments** with protection rules for anything that touches production.

Reuse and efficiency:

- **Matrix** builds run the same job across combinations (OS × language version).
- **Caching** (`actions/cache`) speeds up dependency installs.
- **Artifacts** (`upload-artifact`/`download-artifact`) pass build outputs between jobs and let you download results.
- **Concurrency** cancels superseded runs so you don't waste minutes.

A minimal workflow:

```
yaml
```

```

name: hello
on:
  push:
    branches: [main]
  workflow_dispatch:      # manual trigger button

permissions:
  contents: read          # start least-privilege; add scopes only as needed

concurrency:
  group: hello-${{ github.ref }}
  cancel-in-progress: true

jobs:
  greet:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Say hello
        run: echo "Hello from ${ github.actor } on ${ github.ref_name }"

```

Note the `permissions:` block — always scope the built-in `GITHUB_TOKEN` down to what the workflow actually needs.

Hands-on lab

1. Add the workflow above and trigger it via `workflow_dispatch`.
2. Add a **matrix** over two runtime versions and print each.
3. Add a step that uploads a text file as an **artifact** and download it from the run page.
4. Add a repository **secret** and echo a masked confirmation (never the value) to prove masking works.

Knowledge check

- What is the difference between a variable and a secret?
- What does `concurrency` with `cancel-in-progress` achieve?
- Why scope `permissions:` explicitly instead of relying on defaults?

Chapter 6 — Building CI Pipelines

Objectives

- Build a real continuous-integration pipeline: lint, test, build, report.
- Cache dependencies and gate merges on green checks.
- Factor shared logic into **reusable** and **composite** workflows.

Concepts

Continuous integration means every push and PR is automatically built and tested, so problems surface in minutes rather than at release. A solid CI job typically: checks out code, sets up the toolchain, restores a dependency cache, installs, **lints**, **tests** (with coverage), **builds**, and uploads test/coverage artifacts.

Gating merges. In Chapter 2 you required status checks on `main`. Now make your CI job **be** that status check: its name becomes the required check, so a red build blocks merge automatically.

Reuse across repos and jobs:

- **Composite actions** bundle several steps into one reusable `uses:` action within a repo.
- **Reusable workflows** (`workflow_call`) let many repos call a centrally maintained pipeline — invaluable for keeping a small team's many repos consistent without copy-paste.

Example CI workflow (Node shown; adapt the setup/step commands to your stack):

```
yaml
```

```
name: ci
on:
  pull_request:
  push:
    branches: [main]

permissions:
  contents: read

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'           # built-in dependency caching

      - run: npm ci
      - run: npm run lint
      - run: npm test -- --coverage
      - run: npm run build

      - name: Upload coverage
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: coverage
          path: coverage/
```

A reusable workflow the whole team can call:

yaml

```
# .github/workflows/reusable-ci.yml (in a shared repo)
on:
  workflow_call:
    inputs:
      node-version:
        type: string
        default: '20'
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '${{ inputs.node-version }}', cache: 'npm' }
      - run: npm ci && npm run lint && npm test && npm run build
```

Called from another repo:

```
yaml

jobs:
  ci:
    uses: acme/shared-workflows/.github/workflows/reusable-ci.yml@v1
    with: { node-version: '20' }
```

Hands-on lab

1. Write a CI workflow for your team's actual stack with lint + test + build.
2. Add dependency caching and confirm a cache hit on the second run (compare run times).
3. Make the CI job a **required status check** on `main` via your ruleset.
4. Extract the pipeline into a **reusable workflow** in a shared repo and call it from a second repo.

Knowledge check

- How does a CI job become a merge-blocking gate?
 - When would you choose a composite action vs a reusable workflow?
 - What's the observable sign that dependency caching is working?
-

Chapter 7 — Security & Supply Chain (GitHub Advanced Security)

Objectives

- Enable and act on code scanning, secret scanning, and dependency management.
- Understand keyless authentication with **OIDC** (foundation for Chapter 9).
- Establish a security baseline for the team.

Concepts

GitHub Advanced Security (GHAS) bundles the shift-left security features. Availability depends on licensing (it's included/purchasable depending on your plan), so confirm with your platform owner.

- **Code scanning with CodeQL** — static analysis that finds vulnerabilities (injection, path traversal, etc.) on PRs and pushes. Enable "default setup" for a one-click start, then tune with a workflow if needed. Findings surface in the Security tab and inline on PRs.
- **Secret scanning + push protection** — detects committed credentials (API keys, tokens) and, with **push protection**, blocks the push *before* the secret ever lands in history. This is one of the highest-value features to turn on immediately.
- **Dependabot** — three related capabilities: **alerts** (known vulnerable dependencies), **security updates** (auto-PRs to patch them), and **version updates** (scheduled PRs to keep dependencies current, configured via `.github/dependabot.yml`).
- **Dependency review** — shows on each PR which dependencies changed and whether any introduce known vulnerabilities or problematic licenses.

OIDC and keyless auth (important for deployment). Instead of storing long-lived cloud credentials as secrets, GitHub Actions can present a short-lived, cryptographically signed **OpenID Connect (OIDC)** token that a cloud provider trusts to issue temporary credentials. This removes standing secrets from your repo — the model you'll aim for when connecting Actions to OCI in Chapter 9. To request the token a job needs `permissions: id-token: write`.

A CodeQL default-setup workflow (generated for you, shown for understanding):

```
yaml
```

```

name: codeql
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }
  schedule: [{ cron: '0 3 * * 1' }] # weekly deep scan
permissions:
  contents: read
  security-events: write
jobs:
  analyze:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: github/codeql-action/init@v3
        with: { languages: javascript } # set to your languages
      - uses: github/codeql-action/analyze@v3

```

A Dependabot config:

```

yaml
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: npm
    directory: "/"
    schedule: { interval: weekly }
    open-pull-requests-limit: 5

```

Hands-on lab

1. Enable **secret scanning with push protection** and prove it by attempting to commit a fake-format token.
2. Turn on **CodeQL** default setup and review a finding in the Security tab.
3. Add `dependabot.yml` and merge a resulting version-update PR (after CI passes).
4. Open a PR that adds a dependency and read the **dependency review** summary.

Knowledge check

- What does push protection do that plain secret scanning does not?
- Name the three Dependabot capabilities.

- Why is OIDC preferable to storing a cloud key as a secret?

This chapter touches security topics; apply these features to systems you own or are authorized to work on. Have your platform owner review anything that changes org-wide security posture.

Chapter 8 — Container Builds with Podman

Objectives

- Understand Podman and how it differs from Docker.
- Build OCI-compliant images from a `Containerfile`/`Dockerfile`.
- Run Podman inside GitHub Actions and push images to a registry (**ghcr.io** and **OCIR**).
- Sign and scan images for a trustworthy supply chain.

Concepts

Podman is a daemonless, **rootless** container engine. Unlike Docker, there's no long-running root daemon — each `podman` command runs directly, which is more secure and fits CI runners cleanly. Podman is largely **CLI-compatible with Docker** (`alias docker=podman` works for most flows) and it builds **OCI-format images** by default using **Buildah** under the hood. A `Containerfile` and a `Dockerfile` are the same thing to Podman.

Why Podman for this program. OKE (and Kubernetes generally) runs **OCI images**; the build tool is your choice. Podman's rootless model produces those images without a privileged daemon, which is exactly what you want in ephemeral CI.

A good image is small and reproducible. Use **multi-stage builds** (build in a heavy stage, copy only artifacts into a slim runtime stage), pin base image tags/digests, run as a non-root user, and keep layers cache-friendly.

```
dockerfile
```

```
# Containerfile – multi-stage, non-root, slim runtime
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev
COPY --from=build /app/dist ./dist
USER node # never run as root
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

Registries. Two you'll use:

- **ghcr.io** — GitHub Container Registry, tightly integrated with Actions (authenticate with the built-in `GITHUB_TOKEN`). Great for internal images and PR previews.
- **OCIR** — Oracle Cloud Infrastructure Registry, addressed as `<region-key>.ocir.io/<tenancy-namespace>/<repo>:<tag>`. This is where images destined for OKE typically live so the cluster can pull them within the same tenancy.

Podman is pre-installed on GitHub-hosted `ubuntu-latest` runners. Building and pushing to ghcr.io:

```
yaml
```

```

name: image
on:
  push: { branches: [main] }

permissions:
  contents: read
  packages: write          # needed to push to ghcr.io

jobs:
  build-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build with Podman
        run: podman build -t ghcr.io/${{ github.repository }}:${{ github.sha }} -f Containerfile

      - name: Log in to ghcr.io
        run: echo "${{ secrets.GITHUB_TOKEN }}" | podman login ghcr.io -u ${{ github.actor }}

      - name: Push
        run: podman push ghcr.io/${{ github.repository }}:${{ github.sha }}

```

Trust your images. Two habits worth adopting:

- **Scan** the image for known CVEs (e.g. Trivy or Gype) and fail the build on critical findings.
- **Sign** the image with **cosign** (keyless signing uses the same OIDC identity from Chapter 7) and verify signatures before deploy. Reference images by **digest** (`@sha256:...`), not just mutable tags, so what you scanned and signed is exactly what you deploy.

Hands-on lab

1. Write a multi-stage `Containerfile` for your app; build it locally with `podman build` and run it with `podman run`.
2. Add the Actions workflow above to build and push to **ghcr.io**; confirm the package appears on the repo.
3. Add a **Trivy** scan step that fails on critical vulnerabilities.
4. (Stretch) Sign the pushed image with **cosign keyless** and verify the signature.

Knowledge check

- What does "daemonless and rootless" mean, and why does it matter in CI?
 - Why deploy by image **digest** rather than a tag like `latest`?
 - Which registry would you use for PR-preview images vs images OKE will pull?
-

Chapter 9 — Deploying to OCI OKE with Podman-built Images

Objectives

- Understand OKE and OCIR and how Actions connects to them.
- Choose between **API-key** and **OIDC** authentication to OCI.
- Build a Podman image, push to OCIR, and deploy to OKE from a single pipeline — gated by a GitHub **environment**.

Concepts

Oracle Kubernetes Engine (OKE) is OCI's managed Kubernetes. You deploy standard Kubernetes manifests (or Helm charts) to it; the images come from **OCIR**. The end-to-end pipeline is: *Podman build* → *push to OCIR* → *authenticate to OCI* → *fetch kubeconfig* → `kubectl`/`helm` *apply* → *verify rollout*.

Authenticating Actions to OCI. Two approaches:

1. **API key (baseline, well-documented).** Create an OCI user (ideally a dedicated service user in its own group), generate an **API signing key**, and store the private key + config as GitHub secrets. The OCI CLI in the runner uses them. Simple and reliable; the trade-off is a long-lived credential you must rotate.
2. **OIDC / workload-identity federation (advanced, keyless).** Configure trust in your OCI **identity domain** so it accepts GitHub's OIDC token and exchanges it for a short-lived OCI token. No standing secret. This is the more secure target state (building on Chapter 7) but requires identity-domain setup on the OCI side — plan it with your platform owner and confirm current OCI support before relying on it.

For pushing to OCIR you also need either an **auth token** (username `<tenancy-namespace>/<username>`) or, on the cluster side, an **image-pull secret** so OKE can pull private images:

```
bash
```

```
kubect1 create secret docker-registry ocir-secret \  
  --docker-server=<region-key>.ocir.io \  
  --docker-username='<tenancy-namespace>/<username>' \  
  --docker-password='<auth-token>' \  
  --docker-email='ci@example.com'
```

Reference it in the deployment's `imagePullSecrets`.

Gate production with an environment. Create a GitHub **environment** named `production` with a **required reviewer** (deployment protection rule). The deploy job declares `environment: production`, so a human must approve before the deploy step runs — a lightweight but effective change-control gate for a small team.

A complete build-and-deploy workflow (API-key baseline):

yaml

```
name: build-and-deploy
on:
  push:
    branches: [main]

permissions:
  contents: read
  id-token: write          # reserved for OIDC migration later

env:
  OCIR_REGISTRY: ${vars.OCI_REGION_KEY}.ocir.io
  IMAGE_REPO: ${vars.OCI_NAMESPACE}/myapp

jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      image: ${steps.meta.outputs.image}
    steps:
      - uses: actions/checkout@v4

      - name: Compute image reference
        id: meta
        run: echo "image=${OCIR_REGISTRY}/${IMAGE_REPO}:${GITHUB_SHA::12}" >> "$GITHUB_OUTPUT"

      - name: Build with Podman
        run: podman build -t "${steps.meta.outputs.image}" -f Containerfile .

      - name: Log in to OCIR
        run: |
          echo "${secrets.OCIR_AUTH_TOKEN}" | \
          podman login "$OCIR_REGISTRY" \
          -u "${vars.OCI_NAMESPACE}/${vars.OCIR_USERNAME}" --password-stdin

      - name: Push image
        run: podman push "${steps.meta.outputs.image}"

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment: production          # requires reviewer approval
```

steps:

- **uses:** actions/checkout@v4
- **name:** Install OCI CLI
run: |
curl -L https://raw.githubusercontent.com/oracle/oci-cli/master/scripts/install
| bash -s -- --accept-all-defaults
echo "\$HOME/bin" >> "\$GITHUB_PATH"
- **name:** Configure OCI credentials
run: |
mkdir -p ~/.oci ~/.kube
printf '%s' "\${{ secrets.OCI_CONFIG }}" > ~/.oci/config
printf '%s' "\${{ secrets.OCI_API_KEY }}" > ~/.oci/oci_api_key.pem
chmod 600 ~/.oci/config ~/.oci/oci_api_key.pem
- **name:** Generate kubeconfig for OKE
run: |
oci ce cluster create-kubeconfig \
--cluster-id "\${{ secrets.OKE_CLUSTER_OCID }}" \
--file ~/.kube/config \
--region "\${{ vars.OCI_REGION }}" \
--token-version 2.0.0 \
--kube-endpoint PUBLIC_ENDPOINT
- **name:** Roll out new image
run: |
kubectl set image deployment/myapp \
myapp="\${{ needs.build.outputs.image }}" -n myapp
kubectl rollout status deployment/myapp -n myapp --timeout=180s
- **name:** Rollback on failure
if: failure()
run: kubectl rollout undo deployment/myapp -n myapp

Kubernetes manifest the pipeline updates (store it in-repo and apply with `kubectl apply -f` for first deploy, then let CI `set image`):

yaml

```

apiVersion: apps/v1
kind: Deployment
metadata: { name: myapp, namespace: myapp }
spec:
  replicas: 2
  selector: { matchLabels: { app: myapp } }
  template:
    metadata: { labels: { app: myapp } }
    spec:
      imagePullSecrets:
        - name: ocir-secret
      containers:
        - name: myapp
          image: <region-key>.ocir.io/<namespace>/myapp:placeholder
          ports: [{ containerPort: 3000 }]
          readinessProbe:
            httpGet: { path: /healthz, port: 3000 }

```

Secrets/variables this pipeline expects (set under repo or environment settings): secrets

`OCIR_AUTH_TOKEN`, `OCI_CONFIG`, `OCI_API_KEY`, `OKE_CLUSTER_OCID`; variables `OCI_REGION_KEY`, `OCI_NAMESPACE`, `OCIR_USERNAME`, `OCI_REGION`.

Migrating to keyless (recommended follow-up). Once the baseline works, replace the API-key/config steps with an OIDC token exchange against your OCI identity domain and drop the standing secrets. Keep `permissions: id-token: write` (already present) and confirm current OCI federation guidance before switching production over.

Hands-on lab

1. In OCI: create the OKE cluster (or use an existing one), an OCIR repo, a service user + group, an auth token, and an API key. Record the OCIDs.
2. Add all secrets/variables to a GitHub **environment** called `production` with one required reviewer.
3. Apply the base Deployment manifest and the `ocir-secret` image-pull secret to the cluster once.
4. Push to `main`, approve the environment gate, and watch the image build, push to OCIR, and roll out to OKE. Confirm the running pods use the new digest.
5. Force a bad deploy (e.g. wrong port) and confirm the rollback step recovers the previous version.

Knowledge check

- What are the two ways Actions can authenticate to OCI, and the trade-off between them?
- Why does OKE need an image-pull secret for OCIR?
- What does the `environment: production` gate add to the deploy job?
- How would you migrate this pipeline from API-key to keyless auth?

IAM policies, service users, and cloud credentials are powerful. Scope them to least privilege, work only in tenancies you're authorized to use, and have your platform owner review the OCI-side setup.

Chapter 10 — Operate, Govern & Scale (Capstone)

Objectives

- Round out the feature set: Packages, Codespaces, Copilot, self-hosted runners.
- Apply organization-wide governance and cost controls.
- Complete a capstone that exercises the whole platform.

Concepts

GitHub Packages hosts your artifacts and container images (via `ghcr.io`) alongside code, with the same permission model — useful for internal libraries and image promotion between environments.

Codespaces are cloud development environments defined by a `.devcontainer/`. New team members get a fully configured environment in minutes with no "works on my machine" drift. Set spending limits and idle timeouts to control cost.

GitHub Copilot (and Copilot Enterprise features like PR summaries and knowledge bases, subject to licensing) accelerates coding and review. Agree on team norms: Copilot suggests, humans remain accountable, and generated code goes through the same CI and review gates as everything else.

Self-hosted runners let workflows run on your own infrastructure — necessary when a job needs private-network access (e.g. an OKE control plane on a private endpoint) or specialized hardware. For Kubernetes-native scaling, the **Actions Runner Controller (ARC)** runs ephemeral runners as pods *inside* OKE, which also gives those jobs in-cluster network access for deployments. Secure self-hosted runners carefully: prefer ephemeral runners, never use them on public repos, and keep them patched.

Governance and cost. Use **enterprise/organization policies** to standardize: which Actions are allowed (allow-list trusted publishers), required workflows across repos, SSO/2FA enforcement, IP allow lists, and default permissions for the `GITHUB_TOKEN`. Watch **Actions minutes**, storage, and Codespaces spend in the billing dashboard and set limits so a runaway

workflow can't surprise you. The **audit log** and, where available, audit-log streaming feed your compliance and monitoring.

Capstone project (whole team, end-to-end). Ship a small service from empty repo to running on OKE, exercising every prior chapter:

1. **Repo & governance (Ch 1-2):** new repo in your org, ruleset on `main`, CODEOWNERS, teams assigned.
2. **Planning (Ch 4):** a Project board with the capstone tasks and a milestone.
3. **Collaboration (Ch 3):** every change lands via PR with review and squash-merge.
4. **CI (Ch 5-6):** lint/test/build workflow as a required check; extract it into a reusable workflow.
5. **Security (Ch 7):** CodeQL, secret scanning + push protection, and Dependabot all green.
6. **Image (Ch 8):** multi-stage `Containerfile` built with Podman, scanned, and pushed.
7. **Deploy (Ch 9):** build → OCIR → OKE via the environment-gated pipeline, with working rollback.
8. **Operate (Ch 10):** a `.devcontainer` so anyone can spin up a Codespace; billing limits set; consider ARC runners in OKE as a stretch goal.

Demo it: open a PR that changes a visible string, watch CI + security run, get it reviewed and merged, approve the production gate, and see the change live on OKE.

Team cheat sheet

text

```
Branch & PR:  git switch -c feat/x → push → open PR → review → squash-merge
Run a workflow manually:  Actions tab → select workflow → Run workflow
Build image:  podman build -t <registry>/<ns>/<app>:<tag> -f Containerfile .
Push image:   podman login <registry> && podman push <registry>/<ns>/<app>:<tag>
OKE kubeconfig: oci ce cluster create-kubeconfig --cluster-id <ocid> --file ~/.kube/config
Deploy:       kubectl set image deployment/<app> <app>=<image> -n <ns>; kubectl rollout restart deployment/<app> -n <ns>
Rollback:     kubectl rollout undo deployment/<app> -n <ns>
Debug a pod:  kubectl get pods -n <ns>; kubectl describe pod <p> -n <ns>; kubectl logs <p> -n <ns>
```

Knowledge check

- When do you need self-hosted runners, and what does ARC add on OKE?
 - Name two organization policies worth enforcing on day one.
 - Which cost dimensions should a small team monitor, and how do you cap them?
-

Appendix A — Prerequisites & environment

- A **GitHub Enterprise** account with at least one organization and admin access for the platform owner.
- An **OCI tenancy** with permissions to create OKE, OCIR, users, groups, and policies.
- Local tooling per engineer: `git`, `podman`, `kubect1`, and the `oci` CLI.
- Your identity provider details for SSO/SCIM (Chapter 1).
- Decide your license posture up front: Enterprise Cloud vs Server, whether **GHAS** and **Copilot** are in scope.

Appendix B — Facilitation guide

- 10 weeks at one chapter/week, or 5 weeks pairing (1+2), (3+4), (5+6), (7+8), (9+10). Chapter 9 deserves its own session regardless.
- Rotate the lab facilitator each week; keep the platform owner constant for Chapters 1, 7, 9, 10.
- Track completion on a GitHub Project board (dogfooding Chapter 4).
- Keep a shared **Discussion** for questions and decisions throughout.

Appendix C — Where to verify (things that change often)

Feature availability, exact CLI flags, and UI paths drift between releases and depend on your license tier. Before relying on any step, confirm against the current official documentation for: GitHub Actions, GitHub Advanced Security, rulesets, Projects (v2), OKE, OCIR, OCI CLI, and OCI identity-domain / OIDC federation. Treat OIDC-to-OCI federation in particular as version-sensitive and validate current support with your platform owner.

End of program. Adapt the language/tooling examples (Node is used illustratively) to your team's actual stack.